

CREATE A CLOUDFORMATION STACK FOR YOUR FLASK APP INFRASTRUCTURE WITH THE COMMAND LINE

We are going to create the **infrastructure** for our **Flask Application (Employee Directory Application)** using the **command line**. We are going to **create this infrastructure** with **CloudFormation** using the **IAM** user **onyema_2** from the command line.

I) SEARCH FOR **WSL** ON YOUR LAPTOP AND OPEN IT.

II) **LOG IN** WITH THE USER **onyema_2**. TYPE THE COMMANDS BELOW.

`aws --version` **# confirm you have aws installed locally.**

`which aws` **# check where it is located on your local machine.**

`aws configure` **# log in with your IAM user, input the access and secret keys,**
and region of that user.

`aws sts get-caller-identity` **# confirm your user login identity**

`aws s3 ls` **# test that your user can perform API calls**

III) CREATE THE **PARAMETERS JSON** FILE FOR **CLOUDFORMATION** PARAMETERS OR EDIT THE **7)_parameters.json** FILE PROVIDED FOR YOU.

ALTERNATIVELY, YOU CAN CREATE YOUR PARAMETERS FILE IN YOUR HOME DIRECTORY AND FOLLOW THIS PROCESS BY TYPING THE BELOW.

`nano parameters.json` **# this will create your parameters.json file**

INSIDE THE **PARAMETERS** FILE, CHANGE THE SECTIONS BELOW. MAKE SURE YOU INPUT YOUR OWN **ACCOUNT** AND **REGION** IDS.

[

```
{
  "ParameterKey": "AmiId",
  "ParameterValue": "ENTER YOUR OWN ACCOUNT AND REGION EC2 AMI"
},
{
  "ParameterKey": "InstanceType",
  "ParameterValue": "t3.micro"
},
{
  "ParameterKey": "DBUsername",
  "ParameterValue": "admin"
},
{
  "ParameterKey": "DBPassword",
  "ParameterValue": "Admin12345678"
},
{
  "ParameterKey": "DBName",
  "ParameterValue": "Employees"
},
{
  "ParameterKey": "DBInstanceClass",
  "ParameterValue": "db.t3.micro"
},
{
  "ParameterKey": "VpcId",
  "ParameterValue": "vpc-xxxxxxxxxxxxxxxxxx"
},
{
  "ParameterKey": "SubnetId",
  "ParameterValue": "subnet-xxxxxxxxxxxxxxxxxx"
},
{
  "ParameterKey": "DBSubnetIds",
  "ParameterValue": "subnet-xxxxxxxxxxxxxxxxxx,subnet-yyyyyyyyyyyyyyyyyy"
},
{
  "ParameterKey": "MultiAZDeployment",
  "ParameterValue": "false"
},
{
  "ParameterKey": "ExistingS3BucketName",
  "ParameterValue": "my-photo-albums-123"
}
]
```

1) EC2 INSTANCE AMI

```
{  
  "ParameterKey": "AmiId",  
  "ParameterValue": "ENTER YOUR OWN ACCOUNT AND REGION EC2 AMI"  
}
```

TYPE THE COMMANDS BELOW TO EXTRACT YOUR OWN AMI ID

```
aws ec2 describe-images \  
  --owners amazon \  
  --filters "Name=name,Values=al2023-ami-2023*" \  
           "Name=architecture,Values=x86_64" \  
           "Name=virtualization-type,Values=hvm" \  
  --query 'Images | sort_by(@, &CreationDate) | [-1].[ImageId,Name,CreationDate]'  
 \  
  --output table \  
  --region eu-north-1
```

2) VPC ID

```
{  
  "ParameterKey": "VpcId",  
  "ParameterValue": "vpc-xxxxxxxxxxxxxxxxxx"  
}
```

TYPE THE COMMANDS BELOW TO EXTRACT YOUR DEFAULT VPC ID

```
aws ec2 describe-vpcs \  
  --query 'Vpcs[*].[VpcId,CidrBlock,Tags[?Key==`Name`].Value|[0]]' \  
  --output table
```

3) SUBNET ID FOR EC2 INSTANCE

```
{
```

```
{
  "ParameterKey": "SubnetId",
  "ParameterValue": "subnet-xxxxxxxxxxxxxxxxxx"
}
```

TYPE THE COMMANDS BELOW TO EXTRACT YOUR SUBNET IDS. CHOOSE A PUBLIC SUBNET ID WITH AN AUTOASSIGNED PUBLIC IP WHERE YOUR EC2 INSTANCE WILL LAUNCH.

```
aws ec2 describe-subnets \
  --region eu-north-1 \
  --query 'Subnets[*].{SubnetId:SubnetId, VPC:VpcId, AZ:AvailabilityZone,
CIDR:CidrBlock, AutoAssignPublicIP:MapPublicIpOnLaunch,
Name:Tags[?Key==`Name`.Value|[0]]}' \
  --output table
```

4) SUBNET IDS FOR RDS INSTANCES

```
{
  "ParameterKey": "DBSubnetIds",
  "ParameterValue": "subnet-xxxxxxxxxxxxxxxxxx,subnet-yyyyyyyyyyyyyyyyyy"
}
```

TYPE THE COMMANDS BELOW TO EXTRACT YOUR SUBNET IDS. CHOOSE THE PRIVATE SUBNET IDS WITHOUT AN AUTOASSIGNED PUBLIC IP OR SUBNETS NAMED RDS PVT SUBNET 1, 2, OR 3 WHERE YOUR RDS INSTANCES WILL LAUNCH.

```
aws ec2 describe-subnets \
  --region eu-north-1 \
  --query 'Subnets[*].{SubnetId:SubnetId, VPC:VpcId, AZ:AvailabilityZone,
CIDR:CidrBlock, AutoAssignPublicIP:MapPublicIpOnLaunch,
Name:Tags[?Key==`Name`.Value|[0]]}' \
  --output table
```

AFTER MAKING YOUR **CHANGES TO THE PARAMETERS TEMPLATE**, SAVE YOUR CHANGES IN YOUR **CHOSEN IDE**.

ALSO OBSERVE YOUR **CLOUDFORMATION TEMPLATE**.

7)_application_infrastructure_deployment_CLI.yml

ALTERNATIVELY, YOU CAN CREATE YOUR **CLOUDFORMATION YAML FILE** IN YOUR **HOME DIRECTORY** AND **FOLLOW THIS PROCESS BY TYPING THE BELOW**.

nano application_infrastructure_deployment_CLI.yml # this will create your yml file

IV) **COPY AND PASTE YOUR PARAMETERS TEMPLATE JSON FILE 7)_parameters_file.json AND APPLICATION INFRASTRUCTURE DEPLOYMENT CLI YAML FILES**

7)_application_infrastructure_deployment_CLI.yml IN YOUR HOME DIRECTORY WHERE YOU ARE **LOGGED** INTO **WSL** FROM.

IN YOUR TERMINAL, TYPE THE COMMAND BELOW TO CONFIRM THAT YOU HAVE THESE FILES.

ls # list all the files you have in your home directory

V) **DEPLOY YOUR STACK** BY TYPING THE COMMANDS BELOW.

```
aws cloudformation create-stack \  
  --stack-name employee-application-infrastructure \  
  --template-body file:///7)_application_infrastructure_deployment_CLI.yml' \  
  --parameters file:///7)_parameters_file.json' \  
  --capabilities CAPABILITY_IAM \  
  --region eu-north-1 \  
  --on-failure DELETE
```

THE ABOVE COMMANDS WILL **PRINT A JSON RESULT BELOW** TO SHOW THAT YOUR **STACK WAS SUCCESSFULLY CREATED**.

```
{  
  "StackId": "arn:aws:cloudformation:eu-north-1:140023390772:stack/employee-  
application-infrastructure/dfeeeef0-5842-11f1-9e0c-0a18103e4915",  
  "OperationId": "dff09d90-5842-11f1-9e0c-0a18103e4915"  
}
```

VI) **CONFIRM** THAT YOUR STACK **WAS CREATED SUCCESSFULLY** BY TYPING THE COMMANDS BELOW.

```
aws cloudformation describe-stacks \  

```

```
--stack-name employee-application-infrastructure \  
--region eu-north-1 \  
--query "Stacks[0].StackStatus" \  
--output text
```

IF SUCCESSFUL, YOU SHOULD GET THE RESPONSE, `CREATE_COMPLETE`

MOST TIMES, THE RESOURCES MIGHT NOT FINISH CREATING WHEN WE TYPE THE ABOVE COMMANDS, WHICH MIGHT SHOW `CREATE_IN_PROGRESS`.

TO ALLOW OUR TERMINAL TO WAIT UNTIL ALL YOUR RESOURCES ARE CREATED, TYPE THE COMMANDS BELOW.

```
aws cloudformation wait stack-create-complete \  
--stack-name employee-application-infrastructure \  
--region eu-north-1
```

THE COMMAND ABOVE `PAUSES YOUR TERMINAL` UNTIL THE `STACK FINISHES CREATING ALL THE RESOURCES IN YOUR TEMPLATE`, THEN RETURNS `EXIT CODE 0 (SUCCESS)` OR `255 (FAILURE)`.

VII) CHECK THAT YOUR RESOURCES ARE ALL CREATED. TYPE THE COMMANDS BELOW.

```
aws ec2 describe-instances --output table           # check your EC2 instance
```

```
aws rds describe-db-instances --output table        # check your RDS instance
```

VIII) DELETE YOUR STACK. TYPE THE COMMANDS BELOW.

```
aws cloudformation delete-stack \  
--stack-name employee-application-infrastructure \  
--region eu-north-1
```

IX) CHECK THAT YOUR **RESOURCES ARE ALL DELETED**. TYPE THE COMMANDS BELOW.

aws rds describe-db-instances **# check your RDS instance**

aws ec2 describe-instances **# check your EC2 instance**

X) LOG OUT THE USER **onyema_2**. TYPE THE COMMANDS BELOW.

rm ~/.aws/credentials

nano ~/.aws/config

remove the default section by typing the below and save your file

[default]

region = eu-north-1

aws sts get-caller-identity **# confirm the user onyema_2 is logged out.**